

INT531

วิศวกรรมความน่าเชื่อถือของระบบ

Toil, Automation และ Infrastructure as Code

สัปดาห์ที่ 8 · บ่ายนี้ Lab 6: สร้าง VM ของ 20 กลุ่มด้วยโค้ด

คณะเทคโนโลยีสารสนเทศ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

บรรยาย 1.5 ชม. + ปฏิบัติการ 1.5 ชม. | ปรับปรุงเนื้อหา พ.ศ. 2569 (2026)

```
$ tofu plan
```

เขียนครั้งเดียว
ได้ 20 เครื่องเหมือนกัน

```
# proxmox_vm_qemu.group["g01"] will be created
+ resource "proxmox_vm_qemu" "group" {
  + name     = "int531-g01"
  + cores    = 4
  + memory   = 8192
}
```

```
# ... 19 more
```

Plan: 20 to add, 0 to change,
0 to destroy.

แผนการเรียนรู้วันนี้

เช้า เข้าใจว่าอะไรควรทำอัตโนมัติ และทำอะไรไม่ให้พังทั้งระบบ · บ่าย ลงมือเขียนจริง

01

Toil คืออะไร และวัดอย่างไร

นิยามหกข้อ เพดาน 50% และการคำนวณคัมทุน

02

บันไดห้าขั้นของการทำอัตโนมัติ

ยิ่งขึ้นสูงยิ่งเร็ว และยิ่งพังได้กว้าง

03

หลักการของ Infrastructure as Code

Declarative, Idempotency, Immutable, State และ Drift

04

ตัวอย่างแอปและเครื่องมือ

OpenTofu/Terraform กับ Proxmox และ Ansible Playbook

05

งานวิจัยและกรณีศึกษา

สิ่งที่โค้ดจริงนับหมื่นไฟล์และเหตุการณ์จริงบอกเรา

06

Lab 6 — Infrastructure as Code

สร้าง ทำให้เป็ียง ตรวจสอบ ตั้งค่า แล้วสร้างใหม่ให้เหมือนเดิม

ผลลัพธ์การเรียนรู้ประจำคาบ

เมื่อจบคาบนี้ นักศึกษาจะสามารถ...

SLO 8.1	วัดปริมาณ Toil และจัดลำดับงานที่ควรทำเป็นอัตโนมัติด้วยการวิเคราะห์ความคุ้มค่า	CLO4
SLO 8.2	อธิบาย Declarative, Idempotency, Immutable Infrastructure และการจัดการ State	CLO4
SLO 8.3	เขียนโค้ดสร้างทรัพยากรโครงสร้างพื้นฐาน และการจัดการ State อย่างปลอดภัย	CLO4
SLO 8.4	เขียน Ansible Playbook ที่รันซ้ำได้โดยไม่เปลี่ยนผล	CLO4
SLO 8.5	อธิบายลำดับขั้นตอนของการทำอัตโนมัติ และกลไกป้องกันที่แต่ละขั้นต้องมี	CLO4

01

Toil คืออะไร และวัดอย่างไร

งานที่ทำเสร็จแล้วระบบไม่ได้ดีขึ้น และยิ่งระบบโต งานยิ่งเพิ่ม

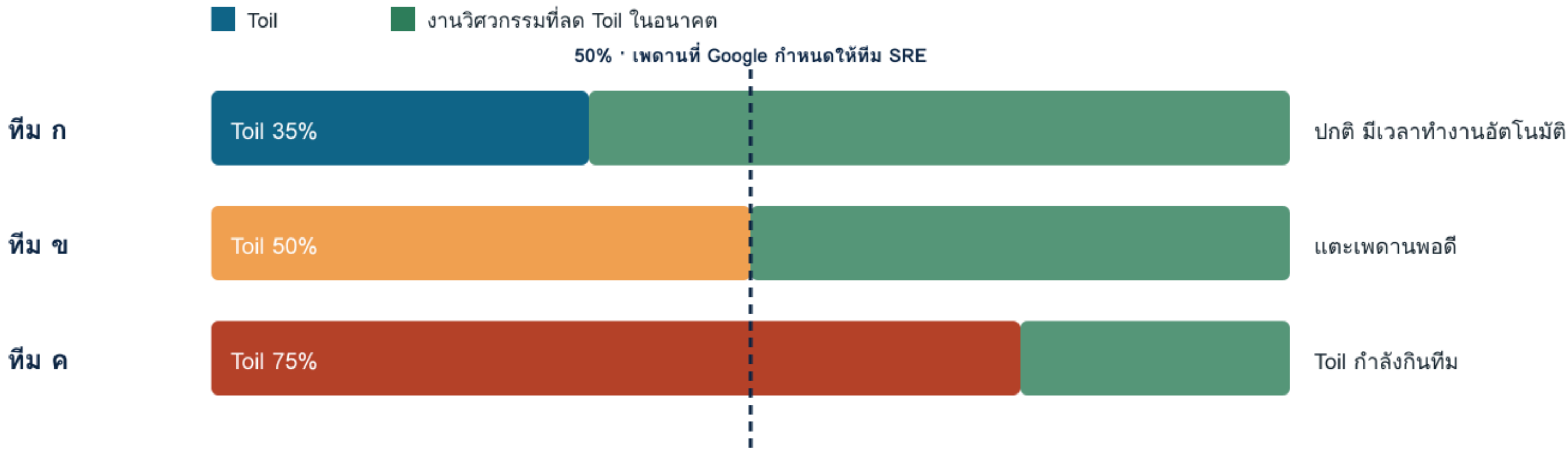
หกลักษณะของ Toil

ยิ่งงานหนึ่งมีครบหลายข้อ ยิ่งเป็น Toil มาก • ตัวอย่างทุกข้อมาจากห้องของเราเอง

ลักษณะ	ความหมาย	ตัวอย่างจากห้องของเรา
Manual	ต้องมีคนลงมือทำเอง	พิมพ์เลขครุภัณฑ์ลงสเปรดชีตที่ละบรรทัด
Repetitive	ทำซ้ำเรื่อย ๆ ไม่ใช่ครั้งเดียว	สร้าง VM ที่ละกลุ่มจนครบ 20 กลุ่ม
Automatable	เครื่องทำแทนได้	ตั้ง VLAN ให้พอร์ตสวิตช์ตามตารางที่มีอยู่แล้ว
Tactical	ตอบสนองต่อเหตุ ไม่ใช่การวางแผน	รีสตาร์ทบริการที่ค้างทุกเช้า
No enduring value	ทำเสร็จแล้วระบบไม่ได้ดีขึ้น	ล้างดิสก์ที่เต็มซ้ำทุกสัปดาห์
Scales with service	ระบบโตเท่าไร งานเพิ่มเท่านั้น	เครื่องเพิ่มหนึ่งเครื่อง งานตั้งค่าเพิ่มหนึ่งชุด

เพดาน 50% — Toil ควรกินเวลาทีมไม่เกินครึ่ง

ตัวเลขจาก Google SRE Book บทที่ 5



ถ้าเกินครึ่ง Toil จะโตเร็วกว่าที่ใครจะทำอัตโนมัติทัน เพราะเครื่องที่เพิ่มขึ้นทุกเครื่องก็เพิ่ม Toil ตามไปด้วย

คำนวณต้นทุน — ให้คะแนนที่วิธีคิด ไม่ใช่ตัวเลขเป๊ะ

ตัวอย่าง: สร้าง VM ของ 20 กลุ่ม ทุกภาคการศึกษา

1	เวลาที่ทำด้วยมือต่อรอบ	ประมาณ 15 นาทีต่อกลุ่ม × 20 กลุ่ม	≈ 5 ชั่วโมง
2	ความถี่	สร้างใหม่ต้นเทอม + แกะระหว่างเทอม	≈ 4 รอบต่อปี
3	เวลาที่เสียต่อปี	5 × 4	≈ 20 ชั่วโมง
4	เวลาลงทุนสร้างโค้ด	เขียน ทดสอบ และทำเอกสาร	≈ 12 ชั่วโมง
5	ต้นทุนดูแลต่อปี	อัปเดตเวอร์ชันและแก้ไขเมื่อเปลี่ยนต้นแบบ	≈ 4 ชั่วโมง

ต้นทุนภายในปีแรก และที่สำคัญกว่าเวลา คือทั้ง 20 เครื่องเหมือนกันแน่นอน ซึ่งการทำด้วยมือรับประกันไม่ได้



บันไดห้าขั้นของการทำอัตโนมัติ

ยิ่งขึ้นสูง Toil ยิ่งลด แต่วงความเสียหายยิ่งกว้าง

จาก Manual ถึง Autonomous

กรณีศึกษาสองเหตุการณ์เกิดขึ้นที่ต่างกัน แต่บทเรียนเดียวกัน



กลไกป้องกันที่สคริปต์ต้องมี

ทำอัตโนมัติโดยไม่มีสิ่งเหล่านี้ คือการทำให้ความผิดพลาดเร็วขึ้น

Dry run ก่อนเสมอ	แสดงว่าจะเปลี่ยนอะไรก่อนลงมือจริง เช่น tofu plan หรือ ansible --check --diff
ขีดจำกัดความเสียหาย	กำหนดจำนวนสูงสุดที่ลบหรือปิดได้ต่อครั้ง เกินกว่านี้ต้องหยุดและถาม
ทยอยทำเป็นรอบ	ทำกับหนึ่งเครื่องก่อน ตรวจสอบผล แล้วค่อยขยาย ห้ามทำทั้งหมดพร้อมกัน
ยืนยันเป้าหมายก่อนลงมือ	แสดงชื่อเครื่องและสภาพแวดล้อมให้เห็นชัด ก่อนคำสั่งที่ย้อนกลับไม่ได้
ทดสอบตัวกลไกป้องกันด้วย	กลไกป้องกันก็คือโค้ดที่มีบั๊กได้ ตามกรณีศึกษาของ Facebook ปี 2564

03

หลักการของ Infrastructure as Code

เขียนว่าต้องการอะไร แล้วให้เครื่องมือหาทางไปถึงเอง

Imperative กับ Declarative

งานเดียวกัน เขียนสองแบบ — สร้าง VM ให้กลุ่มที่ 1

Imperative — บอกขั้นตอน

```
# imperative — steps, in order
qm clone 9000 101 --name int531-g01
qm set 101 --cores 4 --memory 8192
qm set 101 --net0 virtio,bridge=vbr0,tag=101
qm start 101
# ...then repeat 19 more times,
# and hope nobody mistypes an id
```

ต้องจำลำดับเอง ถ้ารันซ้ำจะชนกับเครื่องที่มีอยู่แล้ว และทำ 20 รอบคือโอกาสพิมพ์ผิด 20 ครั้ง

Declarative — บอกสภาพปลายทาง

```
# declarative — the end state
resource "proxmox_vm_qemu" "group" {
  for_each = var.groups
  cores    = 4
  memory   = 8192
  ...
}
# the tool works out what to change
```

เครื่องมือเทียบสภาพปัจจุบันกับที่เขียนไว้ แล้วทำเฉพาะส่วนที่ต่าง รันซ้ำกี่ครั้งก็ได้ผลเดิม

Idempotency — รันซ้ำแล้วไม่มีอะไรเปลี่ยน

ตัวอย่างจริงจาก Ansible: รอบแรกเปลี่ยน 3 อย่าง รอบสองไม่เปลี่ยนแปลง

```
ansible-playbook -i hosts.ini web.yml (run 1, then run 2)

$ ansible-playbook -i hosts.ini web.yml          # run 1
TASK [install nginx] ***** changed: [g01]
TASK [deploy site config] ***** changed: [g01]
TASK [enable and start nginx] ***** changed: [g01]
PLAY RECAP  g01 : ok=4  changed=3  unreachable=0  failed=0

$ ansible-playbook -i hosts.ini web.yml          # run 2
TASK [install nginx] ***** ok: [g01]
TASK [deploy site config] ***** ok: [g01]
TASK [enable and start nginx] ***** ok: [g01]
PLAY RECAP  g01 : ok=4  changed=0  unreachable=0  failed=0

# changed=0 on the second run is the proof of idempotency
```

นี่คือสิ่งที่ **Lab 6** ต้องพิสูจน์ · ถ้ารอบสองยังมี **changed** มากกว่าศูนย์ แปลว่ามืงงานที่ไม่ **Idempotent** ซ่อนอยู่

State และ Drift — เมื่อมีคนไปแก้ด้วยมือ

โค้ดบอกว่า 4 คอร์ 8 GB แต่มีคนเพิ่มในหน้าเว็บ

```
tofu plan (after someone changed a VM by hand)

$ tofu plan
proxmox_vm_qemu.group["g07"]: Refreshing state...

# proxmox_vm_qemu.group["g07"] will be updated in-place
~ resource "proxmox_vm_qemu" "group" {
  name      = "int531-g07"
  ~ memory  = 16384 -> 8192
  ~ cores   = 8 -> 4
}

Plan: 0 to add, 1 to change, 0 to destroy.

# the code says 4 cores / 8 GB – somebody raised it by hand in the web UI
```

State คือบันทึกว่าโค้ดจัดการอะไรอยู่ • ต้องเก็บไว้ที่กลาง มีการล็อก และห้ามเก็บลง Git เพราะอาจมีความลับอยู่ข้างใน

Mutable กับ Immutable Infrastructure

แก้เครื่องที่รันอยู่ หรือสร้างใหม่แล้วสลับแทน

Mutable — แก้ของเดิม

- อัปเดตแพ็คเกจบนเครื่องที่รันอยู่
- แต่ละเครื่องค่อย ๆ ต่างกันไปตามเวลา
- ย้อนกลับยาก เพราะไม่รู้สภาพเดิมแน่ชัด
- เหมาะกับเครื่องที่สร้างใหม่ไม่ได้ง่าย

Immutable — สร้างใหม่

- สร้างเครื่องใหม่จากต้นแบบรุ่นใหม่
- ทุกเครื่องเหมือนกันแน่นอน
- ย้อนกลับคือสลับไปใช้รุ่นก่อน
- ต้องแยกข้อมูลออกจากตัวเครื่องก่อน

กรณี Knight Capital ปี 2555: เครื่องเจ็ตจากแปดเครื่องได้โค้ดใหม่ เครื่องที่แปดไม่ได้ ขาดทุนราว 460 ล้านดอลลาร์ใน 45 นาที



ตัวอย่างแอปและเครื่องมือ

โค้ดที่จะใช้จริงในห้องของเรา

เครื่องมือที่นิยมใช้ แยกตามหน้าที่

สร้างทรัพยากร กับ ตั้งค่าภายในเครื่อง เป็นคนละงานกัน

หน้าที่	เครื่องมือ	ใช้เมื่อ
สร้างทรัพยากร (Provisioning)	OpenTofu, Terraform, Pulumi	สร้าง VM เครือข่าย และดิสก์ ให้มีอยู่ตามที่เขียนไว้
ตั้งค่าภายในเครื่อง (Configuration)	Ansible, cloud-init	ติดตั้งแพ็คเกจ วางไฟล์ตั้งค่า และเปิดบริการ
สร้างต้นแบบเครื่อง (Image)	Packer	สร้าง Template ของ VM ที่ติดตั้งพื้นฐานไว้แล้ว
เก็บความลับ (Secrets)	OpenBao, Vault, SOPS	เก็บรหัสผ่านและคีย์แยกจากโค้ด
ตรวจโค้ดก่อนใช้ (Scanning)	Checkov, tflint, ansible-lint	หากลิ้นอันตรายและข้อผิดพลาดก่อนถึงระบบจริง

เครื่องมือเปลี่ยนเร็ว — ตัวอย่างล่าสุดจาก Terraform

เหตุผลที่รายวิชานี้ใช้ OpenTofu ในแล็บ

10 ส.ค. 2566	HashiCorp เปลี่ยนสัญญาอนุญาตของ Terraform จาก MPL 2.0 ซึ่งเป็นโอเพนซอร์ส ไปเป็น Business Source License 1.1
10 ม.ค. 2567	OpenTofu ซึ่งแยกสายมาจาก Terraform รันก่อนเปลี่ยนสัญญาอนุญาต ออกรุ่นใช้งานจริงรุ่นแรก ภายใต้ Linux Foundation และยังเป็น MPL 2.0
27 ก.พ. 2568	IBM ซื้อกิจการ HashiCorp เสร็จสิ้น มูลค่าประมาณ 6.4 พันล้านดอลลาร์
ผลต่อเรา	สัญญาอนุญาตแบบใหม่ไม่ได้จำกัดการใช้งานภายในองค์กร แต่รายวิชาเลือก OpenTofu เพราะเป็นโอเพนซอร์สเต็มรูปแบบ และใช้ไวยากรณ์เดียวกัน โค้ดในสไลด์นี้ใช้ได้ทั้งสองตัว

ตัวอย่างแอป 1 — VM ของ 20 กลุ่มด้วยโค้ดเดียว

OpenTofu กับ Proxmox · ใช้แผน VLAN 101–120 จากสไลด์เสริมเรื่องเครือข่าย

```
variable "groups" {
  default = { for i in range(1, 21) :
               format("g%02d", i) => 100 + i }
}

resource "proxmox_vm_qemu" "group" {
  for_each      = var.groups
  name          = "int531-${each.key}"
  target_node   = "pve01"
  clone         = "ubuntu-24-template"
  cores         = 4
  memory        = 8192

  network {
    model = "virtio"
    bridge = "vmbr0"
    tag    = each.value      # VLAN 101–120
  }
}
```

for_each

โค้ดชุดเดียวสร้าง 20 เครื่อง ไม่ต้องคัดลอกวาง

tag = each.value

กลุ่มที่ 1 ได้ VLAN 101 ไปจนถึงกลุ่มที่ 20 ได้ VLAN 120 อัตโนมัติ

clone

ทุกเครื่องสร้างจากต้นแบบเดียวกัน จึงเหมือนกันแน่นอน

ไม่มีรหัสผ่านในไฟล์

ข้อมูลเข้าระบบ Proxmox อ่านจากตัวแปรสภาพแวดล้อม ไม่ได้เขียนลงโค้ด

ตัวอย่างแอป 2 — ติดตั้ง Web Server ด้วย Ansible

ทุกงานใช้โมดูลที่ Idempotent ในตัว ไม่ใช่ shell

```
- name: web server for each group
hosts: groups
become: true
tasks:
  - name: install nginx
    ansible.builtin.apt:
      name: nginx
      state: present

  - name: deploy site config
    ansible.builtin.template:
      src: site.conf.j2
      dest: /etc/nginx/sites-enabled/default
      notify: reload nginx

  - name: enable and start nginx
    ansible.builtin.service:
      name: nginx
      state: started
      enabled: true

handlers:
  - name: reload nginx
    ansible.builtin.service: { name: nginx, state: reloaded }
```

state: present

ถ้ามีอยู่แล้ว ไม่ทำอะไร ถ้ายังไม่มี ค่อยติดตั้ง

template

วางไฟล์ตั้งค่าเฉพาะเมื่อเนื้อหาต่างจากเดิม

notify และ handler

รีโหลด nginx เฉพาะเมื่อไฟล์ตั้งค่าเปลี่ยนจริง

ไม่ใช่ shell

คำสั่ง shell ไม่รู้ว่าทำไปแล้วหรือยัง จึงมักทำให้เสีย Idempotency

ตัวอย่างแอป 3 — เส้นทางจากโค้ดถึงระบบจริง

สิ่งที่ทำให้ IaC ปลอดภัยกว่าการคลิกในหน้าเว็บ ไม่ใช่ตัวโค้ด แต่คือการทบทวนก่อนใช้



Pull Request ที่แนบผล plan ไว้ ทำหน้าที่เป็นใบขออนุมัติการเปลี่ยนแปลงได้ในตัว ตามข้อความ **A.8.32** · และมีประวัติว่าใครเปลี่ยนอะไร เมื่อไร เพราะอะไร ครบในที่เดียว

ความลับไม่อยู่ในโค้ด

จากงานวิจัย The Seven Sins (ICSE 2019): พบรหัสผ่านฝังในโค้ด 1,326 จุด อยู่ได้นานสุด 98 เดือน

แบบที่พบบ่อย

```
provider "proxmox" {  
  pm_api_url  = "https://10.20.250.11:8006/api2/json"  
  pm_user     = "root@pam"  
  pm_password = "Int531@2026"      # ← ห้าม  
}
```

แบบที่ถูกต้อง

```
provider "proxmox" {  
  pm_api_url = var.pm_api_url  
  # token อ่านจากตัวแปรสภาพแวดล้อม  
  # PM_API_TOKEN_ID และ PM_API_TOKEN_SECRET  
}
```

และใช้ API Token ที่มีสิทธิ์เท่าที่จำเป็น ไม่ใช่บัญชี root ตรงกับแนวปฏิบัติความปลอดภัยในสไลด์เสริมเรื่อง Virtualization



งานวิจัยและกรณีศึกษา

สรุปจากเอกสารที่แจกไปก่อนคาบ

งานวิจัยที่ควรรู้

วิเคราะห์จากโค้ดจริงหรือเหตุการณ์จริงจำนวนมาก ไม่ใช่ความเห็น

งานวิจัย	สิ่งที่พบ	ใช้กับเรา
The Seven Sins · ICSE 2019	กลั่นแกล้งด้านความปลอดภัย 7 แบบ จากสคริปต์ 15,232 ไฟล์	ห้ามฝังรหัสผ่าน และห้ามเปิดบริการให้ฟังทุกเครือข่าย
Gang of Eight · ICSE 2020	ข้อบกพร่อง 8 ประเภท รวมถึงประเภท Idempotency	Lab 6 ต้องพิสูจน์ Idempotency ด้วยการรันซ้ำ
Ansible และ Chef · TOSEM 2021	ปัญหาเดิมเกิดกับทุกเครื่องมือ	เปลี่ยนเครื่องมือไม่ได้แก้ปัญหาความปลอดภัยให้
Cloud outage study · SoCC 2016	เหตุหยุดทำงาน 597 ครั้งจาก 32 บริการ	ความซ้ำซ้อนไม่ได้กันการตั้งค่าผิดที่ใช้กับทุกชุดพร้อมกัน

ห้ากรณีศึกษา ห้าหลักการ

ทุกกรณีมาจากรายงานที่องค์กรเปิดเผยเอง

เหตุการณ์	เกิดอะไรขึ้น	หลักการที่ขาดไป
AWS S3 · 2560	ใส่พารามิเตอร์ผิดตัวเดียว ถอดเครื่องออกมากกว่าที่ตั้งใจ	ขีดจำกัดความเสียหายในสคริปต์
GitLab · 2560	คำสั่งลบรันบนเครื่องหลักแทนเครื่องสำรอง ข้อมูลหายราว 6 ชั่วโมง	ลดการแตะระบบจริงด้วยมือ และทดสอบการกู้คืน
Knight Capital · 2555	ติดตั้งครบ 7 จาก 8 เครื่อง ขาดทุนราว 460 ล้านดอลลาร์	Immutable และ Reproducible
Facebook · 2564	เครื่องมือตรวจสอบที่ควรหยุดคำสั่งมีบั๊กเอง	ทดสอบตัวกลไกป้องกัน
CrowdStrike · 2567	อัปเดตถึงเครื่องราว 8.5 ล้านเครื่องในเวลาใกล้กัน	ทยอยการเปลี่ยนแปลง



06

Lab 6 — Infrastructure as Code

สร้าง ทำให้เบี่ยง ตรวจสอบ ตั้งค่า แล้วสร้างใหม่ให้เหมือนเดิม

ลำดับงานของ Lab 6

เจ็ดขั้น ทำบนสภาพแวดล้อมทดลองก่อน



แล็บผ่านเมื่อสภาพแวดล้อมที่สร้างใหม่เหมือนครั้งแรกทุกอย่าง ไม่ใช่เมื่อ apply ครั้งแรกสำเร็จ

เกณฑ์ว่าแล็บผ่าน

ทั้งหกข้อต้องแสดงหลักฐานได้

☐	tofu plan แสดงการสร้าง VM ครบตามจำนวนกลุ่มที่กำหนด	ภาพหน้าจอผล plan
☐	หลังแก้ VM ด้วยมือ tofu plan ตรวจพบ Drift ได้	ภาพหน้าจอก่อนและหลัง
☐	Ansible รันรอบสองได้ changed=0	ผลลัพธ์ทั้งสองรอบ
☐	ไม่มีรหัสผ่านหรือ Token อยู่ในไฟล์ที่ commit	ผลตรวจจาก Checkov หรือ tflint
☐	ไฟล์ State ไม่อยู่ใน Git	ไฟล์ .gitignore และที่เก็บ State
☐	ทำลายแล้วสร้างใหม่ได้เหมือนเดิมทุกอย่าง	ผล plan หลังสร้างใหม่ต้องไม่มีการเปลี่ยนแปลง

สถานะห้องจริง: อุปกรณ์ยังอยู่ที่ Staging Area แล็บนี้จึงทำบนสภาพแวดล้อมทดลองก่อน โค้ดชุดเดียวกันจะใช้กับเครื่องจริงเมื่อติดตั้งเสร็จ

การบ้านและสัปดาห์หน้า

สัปดาห์ที่ 9: Containers, Orchestration และ Kubernetes Reliability

งานเดี่ยว ส่งสัปดาห์ที่ 9

- วิเคราะห์ Toil ของตัวเองหนึ่งงาน ตามแบบในเอกสารบทที่ 7
- คำนวณต้นทุน ให้คะแนนที่วิธีคิด
- ระบุความเสี่ยงใหม่ที่เกิดจากการทำอัตโนมัติ และวิธีป้องกัน

งานกลุ่ม ส่งสัปดาห์ที่ 9

- ผลงาน Lab 6 ครบทุกข้อ
- Assignment 2 ทบทวนชุดการแจ้งเตือน
- เสนอหัวข้อโครงงานกลุ่ม

เตรียมสำหรับสัปดาห์หน้า

- ติดตั้ง Docker หรือ Podman ให้พร้อม
- อ่าน Kubernetes Documentation หัวข้อ Pods และ Probes
- คิดว่าถ้า Pod ตายแล้วเกิดใหม่เอง ยังต้องมี Alert หรือไม่

สรุปสามประโยคของวันนี้

01

Toil คืองานที่ทำเสร็จแล้วระบบไม่ได้ดีขึ้น และยิ่งระบบโต งานยิ่งเพิ่ม · ถ้าเกินครึ่งหนึ่งของเวลาทีม ทีมจะไม่มีเวลาเหลือไปแก้มัน

02

การทำอัตโนมัติไม่ได้ลดความเสี่ยงเสมอไป แต่เปลี่ยนรูปแบบของความเสี่ยง · งานมือพลาดทีละเครื่อง งานอัตโนมัติพลาดทีละทั้งระบบ

03

IaC ปลอดภัยกว่าการคลิกในหน้าเว็บ ไม่ใช่เพราะเป็นโค้ด แต่เพราะมีการทบทวนก่อนใช้ ตรวจจับ Drift ได้ และสร้างใหม่ให้เหมือนเดิมได้

สัปดาห์ที่ 9 · Containers และ Kubernetes — เมื่อการสร้างใหม่กลายเป็นเรื่องปกติทุกวัน

เอกสารอ้างอิง

รายการเต็มพร้อมรายละเอียดอยู่ในเอกสารประกอบการเรียนสัปดาห์ที่ 8

ตำราและเอกสารหลัก

- Beyer, B. et al. (2016). Site Reliability Engineering. O'Reilly. บทที่ 5 และ 7
- Beyer, B. et al. (2018). The Site Reliability Workbook. O'Reilly. บทที่ 6
- Morris, K. (2020). Infrastructure as Code (2nd ed.). O'Reilly. บทที่ 1–3
- เอกสารทางการของ OpenTofu, Ansible และ Proxmox VE ฉบับตรงกับรุ่นที่ใช้

งานวิจัยและกรณีศึกษา

- Rahman, Parnin & Williams (2019). The Seven Sins. ICSE 2019
- Rahman, Farhana, Parnin & Williams (2020). Gang of Eight. ICSE 2020
- Gunawi et al. (2016). Why Does the Cloud Stop Computing? SoCC 2016
- รายงานเหตุการณ์ของ AWS, GitLab, SEC (Knight Capital), Meta และ CrowdStrike

หมายเหตุสำหรับผู้สอน

ไม่ต้องฉายในชั้นเรียน

ภาพในเด็คนี้

ภาพทั้งหมดรวมถึงหน้าจอ terminal จำลอง เป็นภาพที่สร้างขึ้นสำหรับรายวิชานี้ ผลลัพธ์ในหน้าจอจำลองจัดรูปแบบให้อ่านง่าย อาจต่างจากผลจริงเล็กน้อยตามรุ่นของเครื่องมือ

โค้ดตัวอย่าง

ชื่อ provider และ attribute ของ Proxmox ต่างกันตามรุ่นของ provider ให้ทดสอบโค้ดกับรุ่นที่จะใช้ในแล็บก่อนสอนเสมอ

ข้อมูลที่อาจเก่า

สถานะสัญญาอนุญาตของ Terraform และ OpenTofu ตรวจสอบเมื่อกันยายน 2569 ให้ตรวจซ้ำทุกภาคการศึกษา